



NAME : HAMDAN ALI

WEEK 4 : Day 16

DAY 16

Select HTML elements and dynamically change text and styles using JavaScript.

Task 16 : Screenshots Of Output

WEEK 4 — DAY 16 • DOM SELECTORS

Live Style Studio

Edit the card on the right. Every change below is one line of JavaScript, shown live in the log.

CONTENT

Title

Hamdan Ali

Role

Full-Stack Developer

Bio

Building clean, dependable interfaces — one component at a time.

APPEARANCE

Accent color

Font

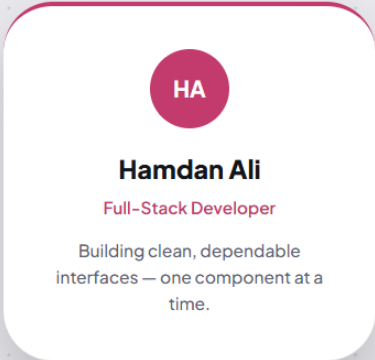
Plus Jakarta Sans

Corner radius 40px

Drop shadow ☒

Dark canvas ☐

Reset Copy HTML





After Doing Changes in text ,style ,Accent ,corner Radius :

RESULT:

WEEK 4 - DAY 16 · DOM SELECTORS

Live Style Studio

Edit the card on the right. Every change below is one line of JavaScript, shown live in the log.

CONTENT

Title

Role

Bio

Building clean, dependable interfaces — one component at a time.

APPEARANCE

Accent color

Font

JetBrains Mono

Corner radius 0px

Drop shadow ☒

Dark canvas ☒

Reset

Copy HTML

SELECTOR LOG

```
document.getElementById("preview-card").classList.toggle("preview-card--shadow", true)
document.getElementById("preview-card").classList.toggle("preview-card--shadow", false)
document.getElementById("canvas-wrap").classList.toggle("canvas-wrap--dark", true)
document.getElementById("canvas-wrap").classList.toggle("canvas-wrap--dark", false)
document.getElementById("preview-card").style.setProperty("--accent", "#FF6B35")
document.getElementById("preview-card").style.setProperty("--accent", "#2F6FED")
```

How it's built

Every control is wired to one specific element, selected once when the page loads and reused from then on — a mix of `document.getElementById()` for elements with a unique id, and `document.querySelector()` / `document.querySelectorAll()` for elements picked out by class or grouped by a shared selector. The log above shows exactly which call ran each time you make a change.

Hamdan Ali · Week 4, Day 16 — Full Stack Web Development Internship



EXPLANATION

DAY 16 — DOM SELECTORS

Building the Live Style Studio

Hamdan Ali · Week 4 · Full Stack Web Development Internship

What I Was Going For

The task for Day 16 was to select HTML elements and change their text and styling with JavaScript — `getElementById` and `querySelector`, basically. That's usually shown as a button that changes one paragraph's color once. I wanted something that actually demonstrated the idea rather than just technically satisfying it, so I built a small live customizer instead: a preview card on one side, a full control panel on the other, and every single control — text fields, color swatches, a font picker, a corner-radius slider, toggles — updates the card instantly through a specific, deliberate DOM selection.

I also added something most versions of this exercise skip entirely: a running log underneath that prints the exact line of JavaScript each action just triggered. Typing in the title field doesn't just change text quietly in the background — it shows you, live, that `document.getElementById("preview-title").textContent` was just set. It turns the concept into something you can watch happen instead of something buried inside a function.

Selecting Everything Once, Up Front

Every element the script touches is looked up a single time at the top of the file and kept in a constant, rather than being re-selected inside each event handler. That's a small habit, but it matters — querying the DOM repeatedly on every keystroke is wasteful, and it's an easy mistake to make when a project starts small and grows.

```
const titleInput = document.getElementById('control-title');
const previewTitle = document.getElementById('preview-title');
const previewRole = document.querySelector('#preview-card .preview-role');
```

```
const swatchButtons = document.querySelectorAll('#swatch-row .swatch');
```

I used `getElementById` for anything with a unique id — which is most of the controls — since it's the more direct lookup when you already know exactly which single element you want. `querySelector` shows up for the role and bio text inside the card, deliberately selected by a class instead of an id, to make the difference between the two approaches visible in real, working code rather than just described in a comment. `querySelectorAll` handles the row of color swatches, since that's naturally a group of similar elements rather than one specific one.

Text Changes

Typing in any of the three text fields — title, role, bio — updates the matching part of the card immediately, through a plain input event listener:

```
roleInput.addEventListener('input', () => {  
  previewRole.textContent = roleInput.value;  
  logAction(`document.querySelector('.preview-role').textContent = "${roleInput.value}"`);  
});
```

Nothing complicated here — the point was to keep this part simple and legible, since it's the most direct illustration of "select an element, change its text" that the task asked for.

Style Changes

The appearance controls all follow the same shape: read the control's value, apply it to the card's style, log what happened. The color swatches were the one part that needed a bit more thought, since six buttons all do the same kind of thing:

```
swatchButtons.forEach((swatch) => {  
  swatch.addEventListener('click', () => {  
    swatchButtons.forEach((s) => s.classList.remove('is-active'));  
    swatch.classList.add('is-active');  
    applyAccentColor(swatch.dataset.color);  
  });  
});
```

Rather than writing a separate click handler for each color, I looped over the whole NodeList once and attached the same handler to every button, reading the actual color from a data-color attribute on whichever one was clicked. Adding a seventh color later would mean adding one button in the HTML — nothing in the JavaScript would need to change.

The accent color itself is applied through a CSS custom property rather than by directly overriding individual color values:

```
function applyAccentColor(color) {  
  previewCard.style.setProperty('--accent', color);  
  previewAvatar.style.background = color;  
}
```

That one line updates the accent everywhere it's used across the card's CSS — the top border, the role text, the avatar — instead of having to update each of those individually in JavaScript.

Giving the Log Something Honest to Say

The log panel isn't decorative — every single control, all the way down to the shadow and dark-canvas checkboxes, writes a real line to it describing exactly what just ran:

```
function logAction(codeLine) {  
  const entry = document.createElement('li');  
  entry.textContent = codeLine;  
  logList.insertBefore(entry, logList.firstChild);  
  while (logList.children.length > 6) {  
    logList.removeChild(logList.lastChild);  
  }  
}
```

I capped it at six entries so it doesn't turn into an unreadable wall of text after a few minutes of clicking around — old entries just fall off the bottom as new ones come in.

Making the Result Actually Usable

The one thing I didn't want was for this to be a toy you play with and then close. The "Copy HTML" button grabs the live card's current markup — with whatever text and styling you've set — and copies it to the clipboard, so the card you've built could genuinely be dropped into an email signature or a bio block somewhere else.

```
try {  
  await navigator.clipboard.writeText(markup);  
} catch (err) {  
  // Clipboard API can fail outside a secure context —  
  // fall back to a manual-select textarea instead.  
}
```

The Clipboard API needs a secure context to work, and this page can be opened straight from a folder with no server behind it at all, so I added a fallback: if the clipboard call fails, the code creates a hidden textarea, selects its contents, and tries the older `execCommand('copy')` approach instead. If even that fails, the button just tells you to copy it manually rather than pretending nothing went wrong.

A Few Notes on What I Took From This

- Selecting elements once and reusing them is a small discipline that only starts to matter as a page grows — worth building the habit early rather than fixing it later.
- `querySelectorAll` plus a single shared event handler scales a lot better than writing one handler per element, especially for something like a row of color options.
- A CSS custom property turned out to be a cleaner way to control a color used in several places than updating each of those places individually from JavaScript.
- Anything that touches the clipboard needs a fallback — it's one of those browser APIs that quietly doesn't work in certain contexts, and failing silently there would have made the feature feel broken for no obvious reason.

References Used

I used the W3Schools page on HTML DOM elements for the exact syntax and differences between `getElementById` and `querySelector`, and watched the linked video to see element selection used in a live, working example.

Article: [w3schools.com/js/js_htmldom_elements.asp](https://www.w3schools.com/js/js_htmldom_elements.asp)

Video: youtube.com/watch?v=_8o_BiLAgQM



Files Submitted

- index.html — the studio layout: control panel, live preview card, and selector log.
- style.css — the design-tool canvas styling, including the dot-grid background and both light/dark canvas states.
- script.js — every DOM selection, the text and style update handlers, the selector log, reset, and the clipboard export with its fallback.

THANK YOU!